

Compiling Typed APIs into Context-Free Languages

Breandan Considine, Clemente Pasti, Tim Viera

August 10, 2026

How long are typical source code statements?

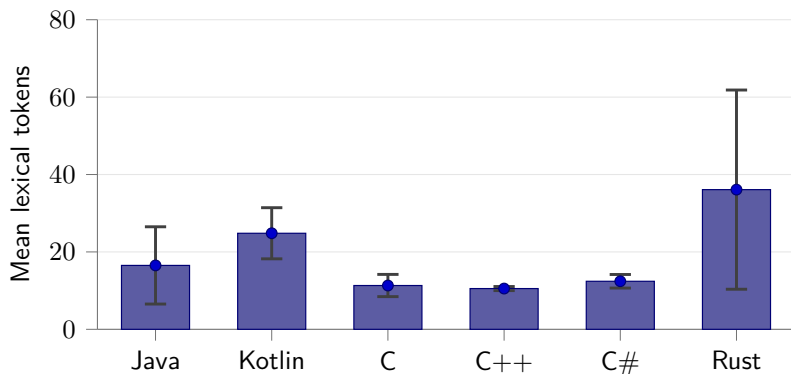


Figure: Whiskers are 95% CIs over sampled files. Statements are Tree-sitter statement nodes with comments and whitespace ignored. Sample size is 100 random GitHub files per language > 20kb.

Motivation

- ▶ Construct a $\ell_\Gamma = \mathcal{L}(G_\Gamma)$ consisting of well-typed statements
- ▶ No well-typed statement $\sigma \notin \ell_\Gamma$ or ill-typed statements $\sigma \in \ell_\Gamma$
- ▶ For any σ such that $\exists \tau. \sigma\tau \in \ell_\Gamma$, display τ 's in shortlex order

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
```

Grammar contribution

$G_{C++} \cup$

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
```

Grammar contribution

$G_{\text{C++}} \cup$

$G_{\text{<string>}} \cup$

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"

using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
}
```

Grammar contribution

$G_{\text{C++}} \cup$

$G_{\text{<string>}} \cup$

$G_{\text{<vector>}} \cup$

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;
// ...

int fizz() { return 3; }
string buzz(string s) { return s; }
// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
```

Grammar contribution

```
GC++ ∪
G<string> ∪
G<vector> ∪
Gorg/fizz.hpp ∪
```

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;
// ...

int fizz() { return 3; }
string buzz(string s) { return s; }
// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
```

Grammar contribution

```
GC++ ∪
G<string> ∪
G<vector> ∪
Gorg/fizz.hpp ∪
Gcom/buzz.hpp ∪
```


Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
```

Grammar contribution

$G_{\text{C++}} \cup$
 $G_{\text{<string>}} \cup$
 $G_{\text{<vector>}} \cup$
 $G_{\text{org/fizz.hpp}} \cup$
 $G_{\text{com/buzz.hpp}} \cup$
 $\{ \text{Type} \rightarrow \text{string} \} \cup$

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
```

Grammar contribution

```
GC++ ∪
G<string> ∪
G<vector> ∪
Gorg/fizz.hpp ∪
Gcom/buzz.hpp ∪
{ Type → string } ∪
...
```

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
```

Grammar contribution

```
 $G_{\text{C++}} \cup$ 
 $G_{\text{<string>}} \cup$ 
 $G_{\text{<vector>}} \cup$ 
 $G_{\text{org/fizz.hpp}} \cup$ 
 $G_{\text{com/buzz.hpp}} \cup$ 
 $\{ \text{Type} \rightarrow \text{string} \} \cup$ 
...
 $\{ \text{Int} \rightarrow \text{fizz()}, \text{S} \rightarrow \text{fizz();} \} \cup$ 
```

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
}
```

Grammar contribution

```
GC++ ∪
G<string> ∪
G<vector> ∪
Gorg/fizz.hpp ∪
Gcom/buzz.hpp ∪
{ Type → string } ∪
...
{ Int → fizz(), S → fizz(); } ∪
{ Str → buzz( Str ), S → buzz( Str ); } ∪
```

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
}
```

Grammar contribution

```
GC++ ∪
G<string> ∪
G<vector> ∪
Gorg/fizz.hpp ∪
Gcom/buzz.hpp ∪
{ Type → string } ∪
...
{ Int → fizz(), S → fizz(); } ∪
{ Str → buzz( Str ), S → buzz( Str ); } ∪
...
```

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
}
```

Grammar contribution

```
GC++ ∪
G<string> ∪
G<vector> ∪
Gorg/fizz.hpp ∪
Gcom/buzz.hpp ∪
{ Type → string } ∪
...
{ Int → fizz(), S → fizz(); } ∪
{ Str → buzz( Str ), S → buzz( Str ); } ∪
...
{ Int → argc, CharPtr → args[ Int ] } ∪
```

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
}
```

Grammar contribution

```
GC++ ∪
G<string> ∪
G<vector> ∪
Gorg/fizz.hpp ∪
Gcom/buzz.hpp ∪
{ Type → string } ∪
...
{ Int → fizz(), S → fizz(); } ∪
{ Str → buzz( Str ), S → buzz( Str ); } ∪
...
{ Int → argc, CharPtr → args[ Int ] } ∪
{ Str → bang } → GΓ
```

Grammar Modularization

C++ context

```
// Filetype: *.cpp
#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;
// ...
int fizz() { return 3; }
string buzz(string s) { return s; }
// ...
int main(int argc, char* args[]) {
    string bang = "...";
    |
}
```

Grammar contribution

```
 $G_{\text{C++}} \cup$ 
 $G_{\text{<string>}} \cup$ 
 $G_{\text{<vector>}} \cup$ 
 $G_{\text{org/fizz.hpp}} \cup$ 
 $G_{\text{com/buzz.hpp}} \cup$ 
 $\{ \text{Type} \rightarrow \text{string} \} \cup$ 
...
 $\{ \text{Int} \rightarrow \text{fizz()}, \text{S} \rightarrow \text{fizz()}; \} \cup$ 
 $\{ \text{Str} \rightarrow \text{buzz( Str )}, \text{S} \rightarrow \text{buzz( Str )}; \} \cup$ 
...
 $\{ \text{Int} \rightarrow \text{argc}, \text{CharPtr} \rightarrow \text{args[ Int ]} \} \cup$ 
 $\{ \text{Str} \rightarrow \text{bang} \} \rightarrow G_{\Gamma}$ 
 $L(G_{\Gamma})$ 
```


Monomorphization

Covariance and contravariance

Results: Size of generated CFGs